

Première connexion SSH

Introduction

SSH (pour « Secure SHell ») permet d'exécuter un shell sur un système qui se trouve à distance de façon sécurisée, voir https://fr.wikipedia.org/wiki/Secure_Shell

C'est un outil fondamental en administration système, car il permet d'administrer des systèmes installés sur des machines distantes sans avoir à se déplacer.

Pour cela, le système distant exécute un *serveur SSH*, qui est un programme qui écoute sur le port 22 (par défaut). Ce service tourne en permanence.

Pour exécuter un shell sur le système distant, vous allez exécuter sur votre système local un *client SSH* qui va ouvrir un port aléatoire sur votre système local et se connecter au port 22 du système distant.

Après vérifications (identification de l'utilisateur avec votre login, authentification avec votre mot de passe, vérification que vous avez l'autorisation de vous connecter), le serveur SSH va vous permettre d'exécuter un shell sur le système distant.

Problématique : MITM

La connexion entre votre système local et le système distant doit être chiffrée pour éviter qu'un attaquant observe ou modifie les commandes que vous envoyez au système distant. Pour cela, le serveur SSH du système distant et le client SSH de votre système local vont établir une clef de chiffrement commune (chiffrement symétrique) à l'aide du protocole de Diffie-Hellman, que vous étudierez en cryptographie en L3.

Mais cela ne suffit pas ! En effet, quel que soit le système de chiffrement, on peut imaginer qu'un système adverse qui se trouve sur le trajet entre votre système local et le système sur lequel vous souhaitez vous connecter :

- se fasse passer pour le système distant auprès de votre système local
- se fasse passer pour votre système local auprès du système distant

et compris lors de l'échange de clefs. Votre système local établit une connexion chiffrée avec l'attaquant en « pensant » le faire avec le système distant, et le système distant établit une connexion chiffrée avec l'attaquant en « pensant » le faire avec le système local. Il suffit alors à l'attaquant de maintenir ces deux connexions en déchiffrant et rechiffrant les échanges qui suivent (tout en pouvant observer ou modifier les messages qui circulent) sans que les systèmes aux extrémités ne s'aperçoivent du problème.

Cette attaque s'appelle l'attaque par l'homme du milieu (en: man in the middle attack, abrégé MITM), voir la page https://fr.wikipedia.org/wiki/Attaque_de_l'homme_du_milieu

Ainsi, il faut trouver un moyen pour le serveur SSH distant de s'authentifier auprès de votre client SSH local, de sorte à ce que vous soyez sûr·e que vous vous connectez bien au bon système et pas à un attaquant qui se trouve sur la route entre les deux systèmes (par exemple votre fournisseur d'accès internet).

Pour cela, le serveur SSH a généré une paire de clefs (clef publique, clef privée) qui sont intimement liées. La clef privée n'est connue que du serveur SSH et il fait parvenir au client la clef publique. Grâce à la clef publique du serveur, le client peut envoyer un « challenge » au serveur, et ce challenge ne pourra être

résolu que si le serveur possède la clef privée correspondant à la clef publique. Grâce à la clef publique, le client peut vérifier que le challenge a été correctement résolu.

Pour s'assurer que le client a bien reçu la bonne clef publique (et pas celle d'un attaquant qui se trouve sur la route), l'utilisateur doit vérifier son hash qu'il a obtenu par un autre moyen. Le hash de la clef publique est appelé *fingerprint* (fr: empreinte digitale).

Idéalement, le fingerprint doit être transmis en main propre.

On échange les fingerprints car ceux-ci sont plus courts que les clefs publiques, et donc plus faciles à vérifier par un être humain.

Première connexion

Le premier système auquel nous allons nous connecter est le système **sercalssh.ig-edu.univ-paris13.fr**. Son nom vient de l'unité qui s'appelait « SERvice de CALcul » à l'institut Galilée, et elle est utilisée comme passerelle SSH pour se connecter aux systèmes des salles TP. Elle est accessible depuis internet (son adresse IP est une adresse publique). Vos identifiants (login et mot de passe) sont les mêmes que ceux vous permettant de vous connecter aux systèmes de salles TP.

Le fingerprint de la clef publique (RSA) du serveur SSH exécuté par **sercalssh.ig-edu.univ-paris13.fr** est

```
SHA256:dN1Wp0xEsZi6joi7KffCwt/VTmxnt/p2iakDe6Qj4IM (RSA)
```

(la liste des fingerprints des serveurs SSH des systèmes de salle TP depuis la page du SI Galilée : <https://si-galilee.univ-paris13.fr/>)

On suppose ici qu'aucun attaquant n'est assez fort (ou assez motivé) pour se trouver sur la route entre votre système local et le serveur SSH de **sercalssh.ig-edu.univ-paris13.fr** et qu'il a pu modifier le fingerprint de cette feuille de TP.

Depuis votre système local, vous pouvez vous connecter à un système distant grâce à la commande :

```
$ ssh <user>@<système_distant>
```

où **<user>** désigne votre nom d'utilisateur sur le système distant et **<système_distant>** désigne soit l'adresse IP du système distant, soit son nom de domaine. La commande **ssh** exécute un client SSH sur votre système local.

1. Connectez-vous en SSH au système **sercalssh.ig-edu.univ-paris13.fr**.
2. Vérifiez que le fingerprint de la clef publique envoyée par le serveur SSH du système distant correspond bien à celui du système **sercalssh.ig-edu.univ-paris13.fr**.
3. Si c'est le cas, acceptez la connexion.
4. Observez que le prompt qui s'affiche sur votre terminal a changé, il devrait être de la forme :

```
<user>@sercalssh:~$
```

Vous voici donc en train d'exécuter un shell sur le système **sercalssh.ig-edu.univ-paris13.fr** à distance.

5. À l'aide des commandes **pwd**, **cd**, **ls**, découvrez quel est votre répertoire utilisateur (**HOME**) sur le système distant, et retrouvez les fichiers que vous avez créés lors de vos TP précédents.
6. Déconnectez-vous du système distant avec la commande **exit**.

Seconde connexion

7. Reconnectez-vous sur le système **sercalssh.ig-edu.univ-paris13.fr** avec SSH, et observez que votre client ne vous a pas demandé de révéifier le fingerprint de la clef publique du serveur SSH.

8. Observez, sur votre système local, qu'un fichier **known_hosts** a été créé dans le répertoire **~/.ssh/**.

Le fichier **~/.ssh/known_hosts** mémorise les fingerprints que vous avez acceptés par le passé, de sorte qu'il n'est pas nécessaire de les révéifier à chaque fois.

Par contre, si votre client SSH se rend compte que le fingerprint d'un système distant a changé, il provoquera une erreur avec un gros warning, qu'il ne faudra pas ignorer !

9. Observez que ce fichier contient l'association système distant -> fingerprint grâce à la commande

```
ssh-keygen -l -f ~/.ssh/known_hosts
```

Ne pas traîner ici

10. Sur le système **sercalssh.ig-edu.univ-paris13.fr**, essayez les commandes **vim**, **nano**, **emacs**, **gcc**, **make**, **git** que vous utilisez habituellement en TP.

Vous constatez que ces logiciels ne sont pas disponibles. En effet, le système **sercalssh** ne doit être utilisée que comme passerelle SSH vers les systèmes de salle TP qui ne sont pas directement reliés à internet. Si tout le monde l'utilisait pour travailler, il ne serait plus possible de se connecter aux systèmes de salle TP par SSH car la passerelle serait saturée. Il ne faut donc pas traîner dessus.

11. Déconnectez-vous du système **sercalssh.ig-edu.univ-paris13.fr**.
12. Lorsque vous avez réussi à vous connecter au système **sercalssh** et compris pourquoi et comment vérifier l'authenticité d'un serveur SSH distant, vous pouvez ajouter **ssh.fingerprint** à vos tags.

Objectifs du TP :

- objectifs opérationnels du TP :
 - se connecter à un système distant par SSH
- objectifs pédagogiques du TP :
 - découvrir SSH
 - se familiariser avec la dualité système local / système distant
 - se familiariser avec la dualité client / serveur
 - comprendre l'authentification du serveur SSH distant à l'aide de la vérification du fingerprint de sa clef publique SSH.